Machine Learning Assignment – Student Independent Task 2

Name: Md Sha Niamatullah

Course: CSCI3234

This notebook performs a complete machine learning workflow including:

- Exploratory Data Analysis
- Data Preprocessing
- Model Training
- Evaluation using multiple metrics

Dataset Used: CDC Diabetes Health Indicators (UCI Repository)

```
# ==============================
# IMPORT LIBRARIES
# ==============================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score,
    roc_curve, precision_recall_curve, auc
)
from sklearn.metrics import average_precision_score

# Install required datasets & model libraries
```

```
# Install required datasets & model libraries
!pip install ucimlrepo
!pip install xgboost

from ucimlrepo import fetch_ucirepo
from xgboost import XGBClassifier


# ==============================
# DATASET LOADING
# ==============================

# Load dataset from UCI repository
dataset = fetch_ucirepo(id=891)

X = dataset.data.features
y = dataset.data.targets

# Combine features and target
df = pd.concat([X, y], axis=1)

# =====================================
# Reduce dataset size for faster training
# =====================================
df = df.sample(n=50000, random_state=42)

# Preview dataset
df.head()
```

```
Requirement already satisfied: ucimlrepo in /usr/local/lib/python3.12/dist-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.1.4)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.3)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	Veggies	...	NoDocbcCost	GenHlth	MentHlth
219620	0	0	1	21	0	0	0	0	1	1	...	0	3	3
132821	1	1	1	28	0	0	0	1	1	1	...	0	3	0
151862	0	0	1	24	0	0	0	1	1	1	...	0	1	0
139717	0	0	1	27	1	0	0	1	0	1	...	0	2	3
239235	0	1	1	31	1	0	0	0	1	1	...	1	4	27

5 rows × 22 columns

Dataset Loading

The dataset was loaded directly from the UCI Machine Learning Repository. A representative random sample was used to reduce computational cost while preserving the overall data distribution.

The dataset was loaded directly from the UCI Machine Learning Repository to ensure authenticity and reproducibility.

DATASET UNDERSTANDING

```
df.shape
```

```
(50000, 22)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 50000 entries, 219620 to 229734
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   HighBP                50000 non-null  int64
 1   HighChol              50000 non-null  int64
 2   CholCheck             50000 non-null  int64
 3   BMI                   50000 non-null  int64
 4   Smoker                50000 non-null  int64
 5   Stroke                50000 non-null  int64
 6   HeartDiseaseorAttack  50000 non-null  int64
 7   PhysActivity          50000 non-null  int64
 8   Fruits                50000 non-null  int64
 9   Veggies               50000 non-null  int64
 10  HvyAlcoholConsump     50000 non-null  int64
 11  AnyHealthcare         50000 non-null  int64
 12  NoDocbcCost           50000 non-null  int64
 13  GenHlth               50000 non-null  int64
 14  MentHlth              50000 non-null  int64
 15  PhysHlth              50000 non-null  int64
 16  DiffWalk              50000 non-null  int64
 17  Sex                   50000 non-null  int64
 18  Age                   50000 non-null  int64
 19  Education             50000 non-null  int64
 20  Income                50000 non-null  int64
 21  Diabetes_binary       50000 non-null  int64
dtypes: int64(22)
memory usage: 8.8 MB
```

```
df.describe()
```

	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	Veggi
count	50000.000000	50000.000000	50000.00000	50000.000000	50000.000000	50000.000000	50000.000000	50000.000000	50000.000000	50000.0000
mean	0.431820	0.425840	0.96214	28.428340	0.444160	0.040520	0.093900	0.754420	0.633460	0.8100
std	0.495335	0.494475	0.19086	6.598534	0.496877	0.197177	0.291692	0.430435	0.481864	0.3922
min	0.000000	0.000000	0.00000	12.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
25%	0.000000	0.000000	1.00000	24.000000	0.000000	0.000000	0.000000	1.000000	0.000000	1.0000
50%	0.000000	0.000000	1.00000	27.000000	0.000000	0.000000	0.000000	1.000000	1.000000	1.0000
75%	1.000000	1.000000	1.00000	31.000000	1.000000	0.000000	0.000000	1.000000	1.000000	1.0000
max	1.000000	1.000000	1.00000	98.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.0000

8 rows × 22 columns

The dataset contains health indicators used to predict diabetes status. Each row represents an individual and features describe medical and lifestyle attributes.

Missing Values

```
df.isnull().sum()
```

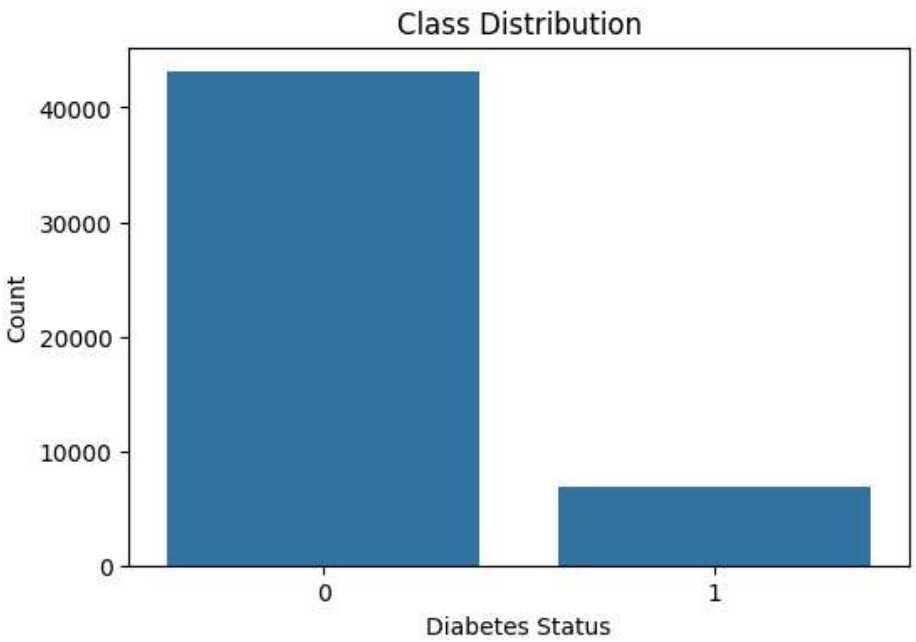
	0
HighBP	0
HighChol	0
CholCheck	0
BMI	0
Smoker	0
Stroke	0
HeartDiseaseorAttack	0
PhysActivity	0
Fruits	0
Veggies	0
HvyAlcoholConsump	0
AnyHealthcare	0
NoDocbcCost	0
GenHlth	0
MentHlth	0
PhysHlth	0
DiffWalk	0
Sex	0
Age	0
Education	0
Income	0
Diabetes_binary	0

dtype: int64

No significant missing values were found.

Class Distribution

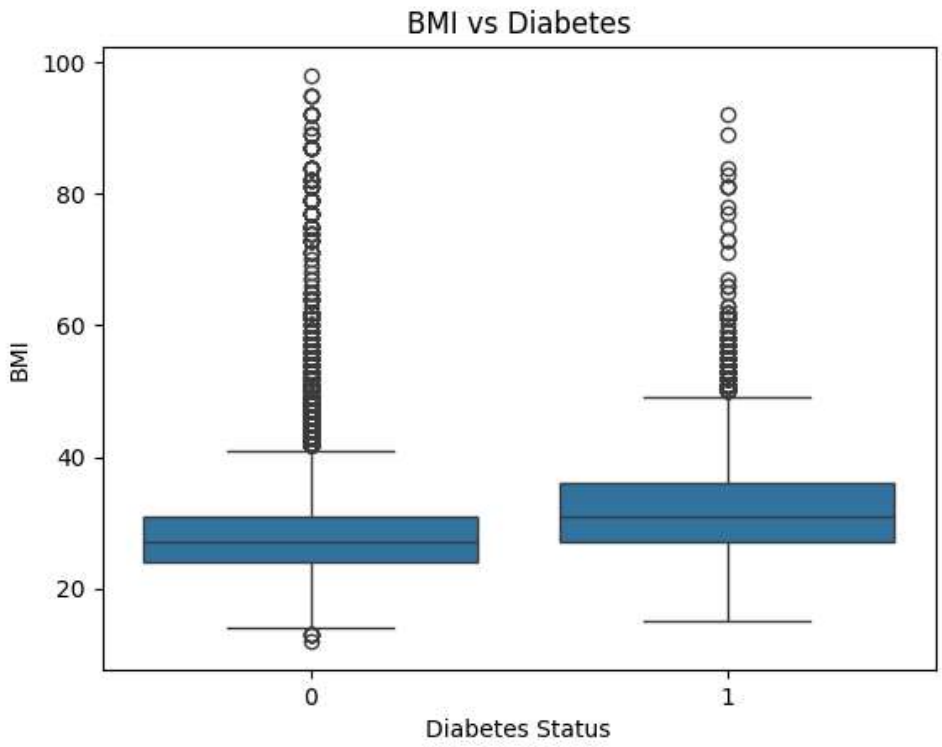
```
plt.figure(figsize=(6,4))
sns.countplot(x='Diabetes_binary', data=df)

plt.title("Class Distribution")
plt.xlabel("Diabetes Status")
plt.ylabel("Count")
plt.show()
```



The dataset shows class imbalance with more non-diabetic samples.

Feature vs Target BMI

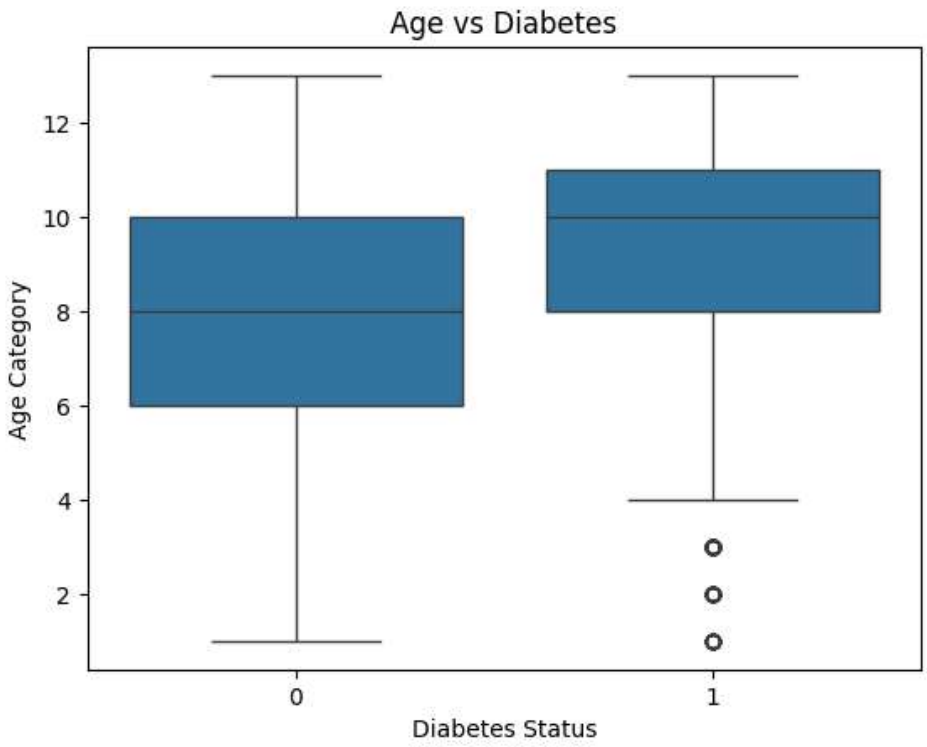
```
sns.boxplot(x='Diabetes_binary', y='BMI', data=df)
plt.title("BMI vs Diabetes")
plt.xlabel("Diabetes Status")
plt.ylabel("BMI")
plt.show()
```



Higher BMI values are associated with diabetes cases.

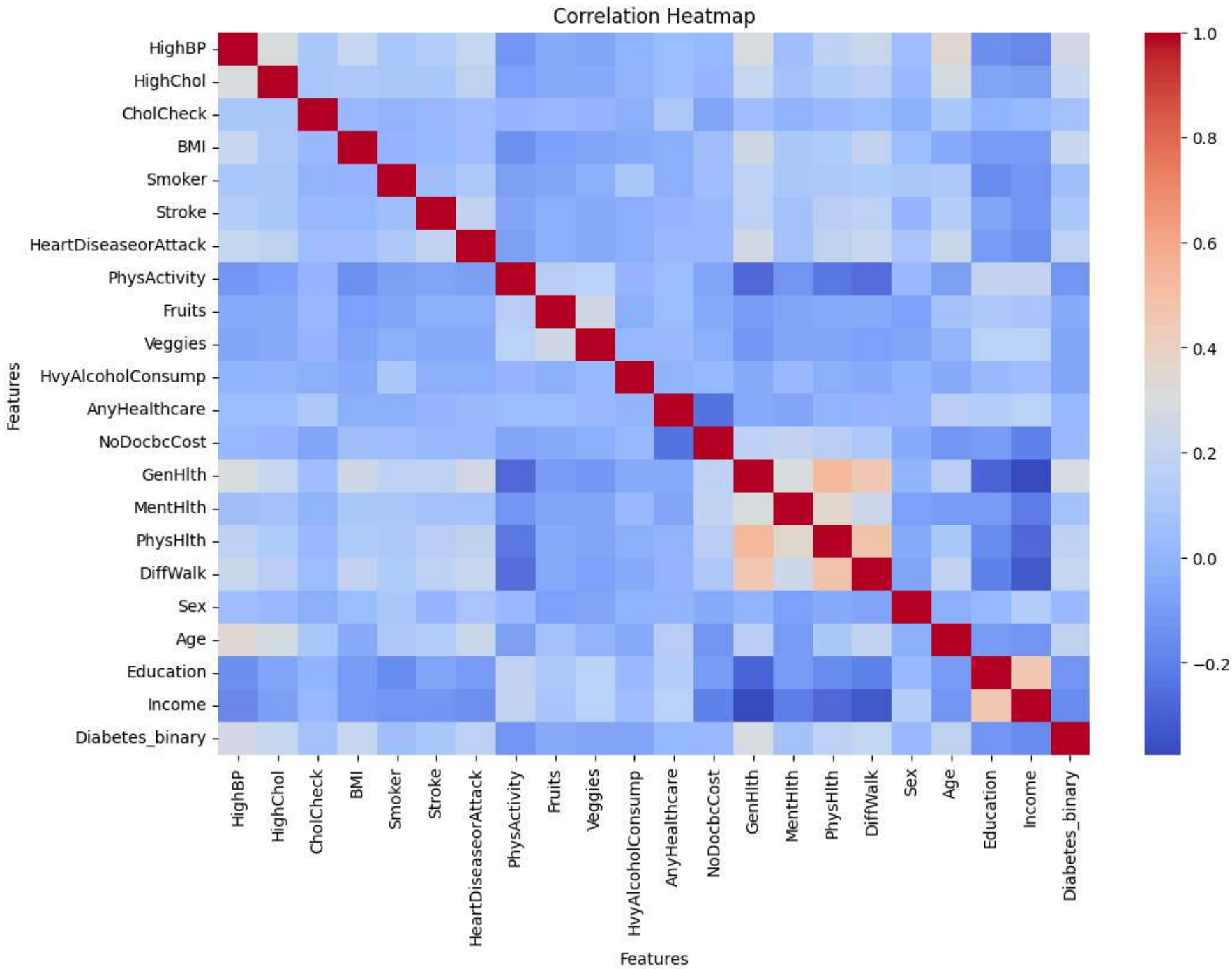
Age

```
sns.boxplot(x='Diabetes_binary', y='Age', data=df)
plt.title("Age vs Diabetes")
plt.xlabel("Diabetes Status")
plt.ylabel("Age Category")
plt.show()
```



Correlation Heatmap

```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.xlabel("Features")
plt.ylabel("Features")

plt.show()
```


Correlation Heatmap
HighBP
HighChol
CholCheck
BMI
Smoker
Stroke
HeartDiseaseorAttack
PhysActivity
Fruits
Veggies
HvyAlcoholConsump
AnyHealthcare
NoDocbcCost
GenHlth
MentHlth
PhysHlth
DiffWalk
Sex
Age
Education
Income
Diabetes_binary
Features
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

The heatmap visualizes correlations between health indicators and the diabetes outcome variable.

DATA PREPROCESSING

Split Features & Target

```
X = df.drop("Diabetes_binary", axis=1)
y = df["Diabetes_binary"]
```

Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Feature scaling was applied to normalize feature ranges for better model performance.

MODEL TRAINING

```
# ==================================================
# SECTION 7 — MODEL TRAINING
# ==================================================

models = {
    "Logistic Regression": LogisticRegression(max_iter=500),
    "Decision Tree": DecisionTreeClassifier(max_depth=10),
    "Random Forest": RandomForestClassifier(n_estimators=40, random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "SVM": SVC(kernel="linear", probability=True),
    "XGBoost": XGBClassifier(eval_metric='logloss')
```

```
}

results = {}
trained_models = {}

for name, model in models.items():

    print(f"Training {name}...")

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # PR-AUC calculation
    pr_auc = average_precision_score(y_test, y_prob)

    trained_models[name] = model

    results[name] = [
        precision_score(y_test, y_pred),
        recall_score(y_test, y_pred),
        f1_score(y_test, y_pred),
        roc_auc_score(y_test, y_prob),
        pr_auc
    ]

results_df = pd.DataFrame(
    results,
    index=["Precision", "Recall", "F1-score", "ROC-AUC", "PR-AUC"]
).T

results_df
```

```
Training Logistic Regression...
Training Decision Tree...
Training Random Forest...
Training KNN...
Training SVM...
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
Training XGBoost...
```

	Precision	Recall	F1-score	ROC-AUC	PR-AUC
Logistic Regression	0.523438	0.144604	0.226607	0.829898	0.419053
Decision Tree	0.456693	0.208633	0.286420	0.783431	0.349196
Random Forest	0.512931	0.171223	0.256742	0.804952	0.382023
KNN	0.405694	0.164029	0.233607	0.742253	0.292867
SVM	0.000000	0.000000	0.000000	0.657532	0.250316
XGBoost	0.548077	0.205036	0.298429	0.824146	0.415818

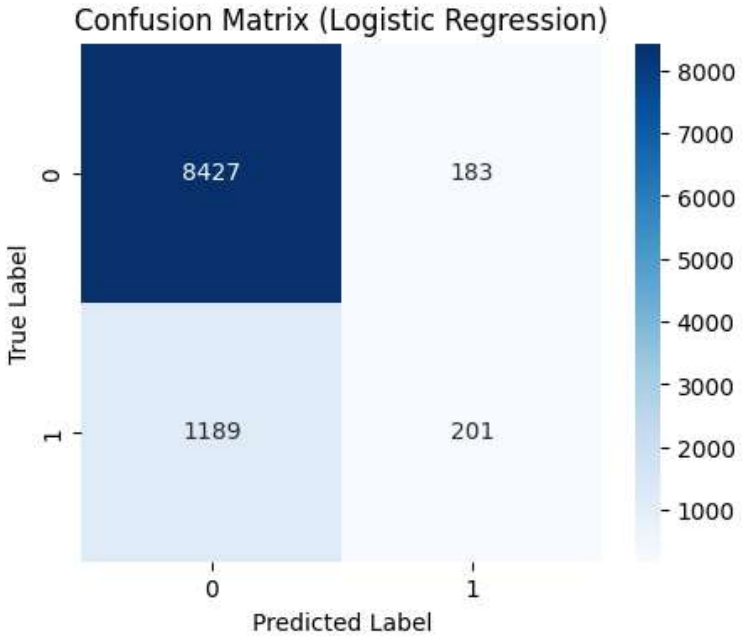
Next steps: Generate code with `results_df` New interactive sheet

Multiple machine learning models were trained and evaluated using Precision, Recall, F1-score, and ROC-AUC metrics to compare their performance for diabetes prediction.

```
# ==================================================
# CONFUSION MATRIX (Best Model)
# ==================================================

best_model_name = results_df["ROC-AUC"].idxmax()
best_model = trained_models[best_model_name]

print("Best Model:", best_model_name)

pred_best = best_model.predict(X_test)

cm = confusion_matrix(y_test, pred_best)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
```

```
plt.title(f"Confusion Matrix ({best_model_name})")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```

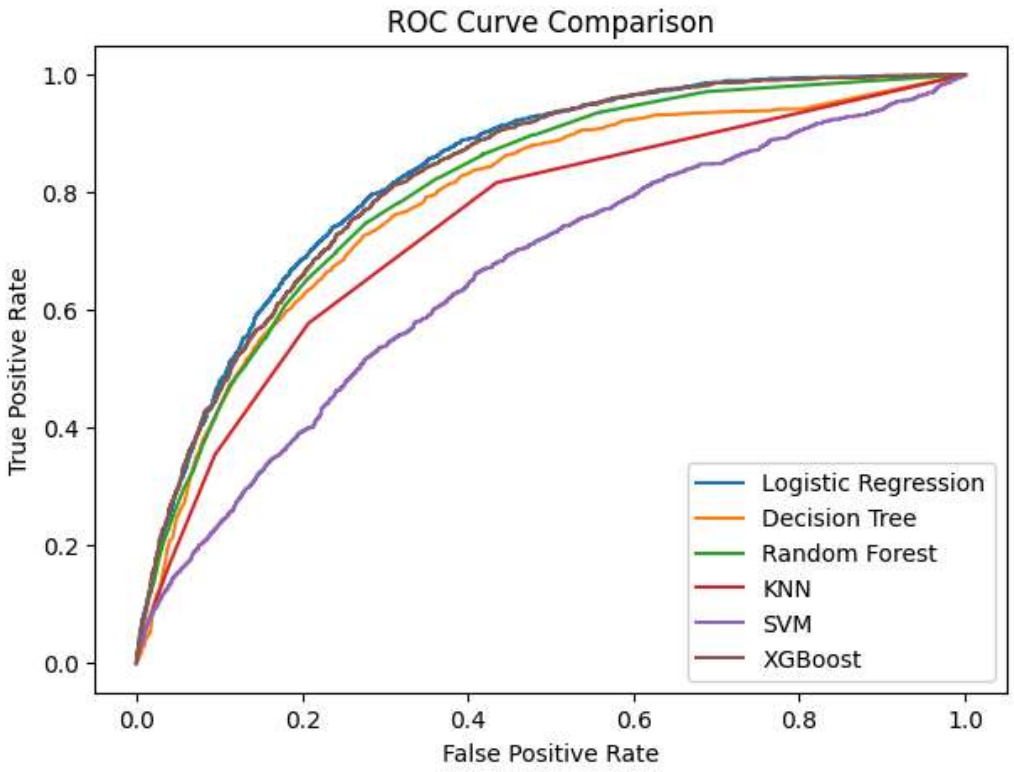
```
Best Model: Logistic Regression
```



The confusion matrix illustrates prediction performance of the best-performing model. Diagonal values represent correctly classified samples.

```
# ==================================================
# ROC CURVE
# ==================================================

plt.figure(figsize=(7,5))

for name, model in trained_models.items():
    prob = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, prob)
    plt.plot(fpr, tpr, label=name)
```

```
plt.title("ROC Curve Comparison")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()

plt.show()
```



The ROC curve compares classification performance across all trained models. Higher curves indicate better discrimination capability.

```
# ==================================================
# PRECISION-RECALL CURVE
# ==================================================

plt.figure(figsize=(7,5))

for name, model in trained_models.items():
    prob = model.predict_proba(X_test)[:,1]
```

```
    precision, recall, _ = precision_recall_curve(y_test, prob)
    plt.plot(recall, precision, label=name)
```